ST. FRANCIS INSTITUTE OF TECHNOLOGY
An Autonomous Institute, Affiliated to University of Mumbai
Department of Computer Engineering

PROJECT DETAILS FOR IPR EVALUATION

VERIQO

An AI-Powered Platform for Hackathon Management, Participant Evaluation, Code Integrity Verification, and Developer Analytics

Submitted by	Aryan Brahmane (Roll No. 46) Verena Fernandes (Roll No. 56) Vedant Gowda (Roll No. 60) Ayush Gurav (Roll No. 61)
Programme	B.E. Computer Engineering, Semester VII — Major Project
Academic year	2026–27
Project guide	Mr. Prasad Pulekar
Submitted to	Project Guide, for forwarding to the Institute IPR Cell
Date	12 October 2026

CONFIDENTIAL — prepared for internal IPR evaluation. Please do not circulate outside the Department and the IPR Cell, as public disclosure may affect patentability.

Contents

1. Project at a Glance 3
2. Summary of the Work 3
3. Problem Addressed 4
4. Proposed Solution and System Architecture 4
4.1 Modules 4
4.2 Architecture 4
5. Development Status and Evidence 5
6. Novel and Inventive Features 5
N1. Fused, time-bounded originality assessment 5
N2. Pre-event code share from version history 5
N3. Privacy-preserving edit-provenance classification in the IDE 6
N4. Event-aware plagiarism corpus 6
N5. Accusation-averse clone classifier 6
7. Related Work and Existing Systems 6
8. IPR Assessment 7
8.1 Copyright 7
8.2 Patent 7
8.3 Illustrative Claims (draft for discussion) 8
8.4 Trademark 8
9. Ownership, Contributions and Disclosures 8
9.1 Inventors / authors and contributions 8
9.2 Third-party components (not claimed) 9
9.3 Disclosure of AI-assisted development 9
9.4 Prior disclosures 9
10. Applications and Requested Action 9
10.1 Potential applications 9
10.2 Request to the IPR Cell 10
Annexure A — Clone Classifier: Data, Method and Results 11
Limitations 11

1. Project at a Glance

Title	VERIQO: An AI-Powered Platform for Hackathon Management, Participant Evaluation, Code Integrity Verification, and Developer Analytics
Domain	Artificial Intelligence, Natural Language Processing, Software Engineering Analytics, Academic/Competition Integrity
Type of work	Software system: web platform, AI microservice and IDE (Visual Studio Code) extension
Current stage	Working prototype. All four modules are implemented, integrated and tested end to end (see Section 5).
Size	Approximately 8,000 lines of original source code across backend, AI service, web frontend and IDE extension; 31 automated tests
Technology	Python (FastAPI), React, PostgreSQL, MongoDB, spaCy, Sentence-Transformers, tree-sitter, scikit-learn, TypeScript (VS Code Extension API)
Available on request	Source code, project report, model evaluation report and a live demonstration
IPR forms proposed	**Copyright** (software) — recommended **Patent** — potential, subject to IPR Cell assessment (Section 6, 8) **Trademark** — for the name "VERIQO", subject to a clearance search

2. Summary of the Work

Hackathons are widely used for innovation and technical recruitment, but organizers cannot easily verify that submitted code was actually written by the team **during** the event. Submissions are static repositories or archives; code can be copied from earlier projects, reused from public repositories, prepared before the event, or produced largely by AI coding assistants. Published research shows that text-based AI-content detectors perform close to chance on source code, so a single detection score cannot be relied upon.

VERIQO addresses this with a **multi-signal originality assessment** that combines three independent kinds of evidence collected over the hackathon's time window: (i) structural and lexical similarity of the submitted code to other submissions and reference repositories, (ii) the share of the final code that already existed in the repository before the event started, derived from version-control history, and (iii) privacy-preserving coding-activity telemetry captured by an IDE extension, which distinguishes typed code from pasted and bulk-inserted code without ever transmitting the code itself. The signals are fused into an explainable risk band (Clear / Review / High risk) with human-readable reasons, presented to judges as evidence rather than an automatic verdict.

The platform also automates participant shortlisting (resume and GitHub analysis with weighted skill matching), forms skill-balanced teams, and provides participants with a verified "Year in Code" developer profile.

3. Problem Addressed

- **Unverifiable effort:** existing hackathon platforms accept a final repository or archive and offer no evidence of when, how or by whom the code was written.
- **Pre-built and recycled projects:** teams can submit work prepared before the event or reused from past hackathons and public repositories.
- **Generative AI:** AI coding assistants make it hard to distinguish authentic effort from generated code, and existing detectors are unreliable on source code.
- **Fragmented tooling:** plagiarism checkers, event platforms and judging are separate; organizers export code manually to third-party tools that do not understand the event's time window, allowed starter code, or team composition.
- **Manual screening at scale:** shortlisting hundreds of applicants against required skills is slow and inconsistent.

4. Proposed Solution and System Architecture

4.1 Modules

Module	Function
A. Shortlisting & team formation	Extracts skills from PDF/DOCX/TXT resumes (rule-based named-entity recognition over a curated 113-skill taxonomy) and public GitHub profiles; scores each applicant against weighted event requirements (exact match, embedding-based semantic match using a pretrained sentence-transformer, and same-category credit); shortlists against an organizer-set threshold; forms skill-balanced teams with a greedy coverage algorithm.
B. IDE telemetry extension	Visual Studio Code extension that records, only inside the designated project folder, counts of typed, pasted and bulk-inserted characters, active coding time and large-insertion events. It never transmits code, file names or paths. Batches are queued offline and sent idempotently.
C. Code integrity engine	Parses code with tree-sitter (14 languages) into normalised token streams and syntax-tree n-grams, computes winnowing fingerprints, compares each submission with all other submissions and reference repositories, excludes organizer-approved starter templates, measures pre-event code from git history, and applies a trained clone classifier.
D. Dashboards & analytics	Organizer evaluation grid with colour-coded, explainable originality reports and file-level evidence; applicant ranking; AI team board; participant "Year in Code" profile.

4.2 Architecture

A React web application communicates with a FastAPI application server. Computationally heavy work (NLP, embeddings, repository analysis, classification) runs in a separate AI microservice so that it cannot block web requests. Relational data (users, events, applications, teams, submissions, reports) is stored in PostgreSQL; telemetry and code fingerprints are stored in MongoDB. The IDE extension communicates with the application server over authenticated JSON requests.

5. Development Status and Evidence

The system is implemented and has been tested end to end against live cloud databases. The figures below are measured, not projected.

Item	Status / measured result
REST API	32 endpoints (authentication, events, applications, teams, submissions, scans, telemetry, reports, analytics)
Automated tests	31 passing (12 backend, 11 AI service, 8 IDE extension)
End-to-end demonstration	Three real public GitHub repositories scanned: an independent project was rated **Clear**; a submission identical to a reference project was rated **High risk** with 98% matched code; a team with a paste-heavy (simulated) telemetry profile was rated **Review**. An unreadable resume was correctly routed to manual review.
Clone classifier (held-out test set, 529 pairs)	Precision 0.982, recall 0.906, F1 0.942, false-positive rate 3.6%, ROC-AUC 0.973
Baseline for comparison (containment ≥ 0.5)	Precision 0.991, recall 0.590, F1 0.740 — the trained classifier detects about 31 percentage points more copies
Fingerprinting speed	≈ 0.15 s for a 2,600-line source file

Training data, method and limitations of the classifier are given in Annexure A. In particular, the benchmark uses synthetic transformations of real code, and the false-positive rate on JavaScript is not yet acceptable; both points are stated so that the evaluation is not overstated.

6. Novel and Inventive Features

Individual building blocks used by VERIQO are known (for example, winnowing fingerprints, cosine similarity and gradient boosting). The team believes the novelty lies in the following features and, above all, in their **combination** for time-bounded coding events. A formal prior-art search has not yet been carried out and is requested from the IPR Cell (Section 10).

N1. Fused, time-bounded originality assessment

Three independent evidence types — code similarity, pre-event code share from version history, and IDE activity telemetry — are evaluated over the same event window and fused into a single risk band with explicit, human-readable reasons. A corroboration rule ensures that telemetry alone can raise a submission to "Review" but never to "High risk"; it can only escalate a case when code-level evidence also exists. This reduces false accusations caused by any one unreliable signal.

N2. Pre-event code share from version history

The repository is evaluated at the last commit made before the submission deadline (later commits are ignored and reported), and a snapshot is reconstructed at the last commit made before the event started. The share of the final submission's fingerprints already present in that pre-event snapshot quantifies how much code was written before the event, independently of any external corpus.

N3. Privacy-preserving edit-provenance classification in the IDE

The extension intercepts the paste command to obtain only the clipboard length, then classifies each subsequent editor change as typed, pasted, bulk-inserted (a large insertion without a paste, characteristic of AI code completion) or same-size replacement (formatting/refactoring, ignored). Only counts, line totals, file extensions and timestamps leave the device; text, file names and paths never do, and tracking is restricted to the designated project folder. Batches carry client-generated identifiers so retries are never double-counted.

N4. Event-aware plagiarism corpus

Every submission is compared with every other submission of the same event and with organizer-supplied reference repositories. Fingerprints of organizer-approved starter templates are removed before matching (so permitted boilerplate is never flagged), fingerprints common to many independent sources are suppressed as generic idioms, and files too small to constitute evidence are excluded by a gate applied identically during training and operation.

N5. Accusation-averse clone classifier

A gradient-boosting classifier over six fingerprint and syntax-tree features is trained with monotonic constraints (more shared code can never lower the clone probability) and its decision threshold is selected by capping the false-positive rate, rather than precision, because precision depends on how common copying is in a given event.

7. Related Work and Existing Systems

System / work	What it does	Difference from VERIQO
MOSS (Stanford); winnowing algorithm (Schleimer, Wilkerson & Aiken, SIGMOD 2003)	Fingerprint-based software similarity detection	Pairwise similarity only; no event time window, version-history analysis, IDE telemetry or template exclusion
JPlag (Prechelt, Malpohl & Philippsen, 2002)	Token-based plagiarism detection for programs	Same as above; not integrated with event management
Hackathon platforms (e.g. Devfolio, Unstop)	Registration, team management, project submission	No built-in code integrity analysis, telemetry or automated skill-based shortlisting (as surveyed in the project report)
"Assessing AI Detectors in Identifying AI-Generated Code" (2024)	Evaluates five AI-content detectors on 5,069 code samples	Evaluation study; finds near-chance accuracy, which motivates VERIQO's use of telemetry instead
"Smart Hackathon Management System with ML Integration" (2025)	Proposal summarisation (Seq2Seq), OCR, public voting	No shortlisting, code integrity analysis or IDE monitoring

System / work	What it does	Difference from VERIQO
"AI-Powered Automated Resume Screening and Job Matching" (2025)	SBERT embeddings and cosine similarity for job matching	Recruitment only; no events, teams or code verification
Keystroke/IDE logging in programming education (research tools)	Records student editing behaviour	To be examined in the formal search; VERIQO's distinguishing points are content-free provenance classification and its fusion with code and history evidence

8. IPR Assessment

Form of IPR	Applicable?	Assessment
Copyright	Yes (recommended)	The source code, documentation and user-interface text are original literary works of the team and can be registered as computer software.
Patent	Possible — to be assessed	Features N1–N4 may qualify as a technical process. Under Section 3(k) of the Patents Act, 1970, a computer programme per se is not patentable, so the application would need to show a technical contribution. Requires a prior-art search and expert opinion.
Trademark	Possible	"VERIQO" is a coined word, which generally favours distinctiveness. Likely classes: 9 (software) and 42 (software as a service). An availability search on the IP India public search portal has not yet been done.
Design registration	Not applicable	The project has no physical product design; the screens of a software interface are not the subject of this disclosure. To be confirmed by the IPR Cell.

8.1 Copyright

- **Subject matter:** source code of the API server, AI service, training pipeline, web frontend and IDE extension; project documentation.
- **Procedure:** application for registration of a computer-software (literary) work through the Copyright Office e-filing portal, with a copy of the source code as prescribed.
- **Note:** third-party open-source components remain under their own licences and are not claimed (Section 9.2).

8.2 Patent

- **Strongest candidate:** the fused, time-bounded originality assessment (N1) incorporating pre-event code share (N2) and privacy-preserving edit-provenance telemetry (N3).
- **Pre-requisites:** a prior-art search of patent and non-patent literature, and an opinion on technical effect under Section 3(k).
- **Novelty protection:** until the IPR Cell advises, the team will keep the repository private and avoid publishing papers, public demonstrations or detailed posts. A provisional application could secure a priority date if filing is recommended.
- **Illustrative claims** are given in Section 8.3 to help the evaluation; they are a starting point for a patent professional, not a final claim set.

8.3 Illustrative Claims (draft for discussion)

Claim 1. A computer-implemented method of assessing the originality of a software submission made during a time-bounded event, comprising: (a) receiving, from an editor extension on a participant's device, edit-activity records restricted to a designated project folder, each record containing counts of characters classified as typed, pasted or bulk-inserted and no source-code content; (b) selecting, from the submission's version history, the last commit made at or before the event end time and computing normalised token fingerprints and syntax-tree n-grams of the source files at that commit; (c) reconstructing the repository at the last commit made before the event start time and computing the proportion of the fingerprints of step (b) already present in that reconstruction; (d) comparing the fingerprints of step (b) with those of other submissions to the same event and of reference repositories, after removing fingerprints of designated permitted templates, and estimating a matched-code share using a trained classifier; and (e) combining the results of steps (a), (c) and (d) into a risk category accompanied by human-readable reasons, wherein the edit-activity evidence of step (a) alone cannot assign the highest risk category.

Claim 2. The method of claim 1, wherein the paste classification is obtained by intercepting the editor's paste command, recording only the length of the clipboard contents, and attributing the next editor change within a predetermined interval to pasting.

Claim 3. The method of claim 1, wherein an editor change exceeding a predetermined size that is not attributed to pasting is classified as bulk-inserted, and a change replacing text of substantially the same length is excluded from all counts.

Claim 4. The method of claim 1, wherein each edit-activity record carries a client-generated identifier and records whose identifier has already been stored are discarded, so that retransmission does not alter the totals.

Claim 5. The method of claim 1, wherein the classifier is a gradient-boosting model constrained to be non-decreasing in each code-overlap feature and whose decision threshold is the lowest value whose false-positive rate on validation data does not exceed a predetermined limit.

Claim 6. The method of claim 1, wherein fingerprints occurring in more than a predetermined number of independent sources are disregarded, and file pairs sharing fewer than a minimum number of fingerprints, or containing fewer than a minimum number of tokens, are not classified.

Claim 7. A system comprising an editor extension, an application server and an analysis service configured to carry out the method of any of claims 1 to 6.

8.4 Trademark

The team proposes "VERIQO" as the product name. A clearance search for identical or similar marks in classes 9 and 42 should precede any application. A logo has not yet been finalised.

9. Ownership, Contributions and Disclosures

9.1 Inventors / authors and contributions

The work was carried out by the four students named on the cover page under the guidance of Mr. Prasad Pulekar. Individual contributions are recorded below for the IPR Cell's reference; ownership follows the Institute's IPR policy.

Team member	Principal contribution (to be completed by the team)
Aryan Brahmane (46)	[ ]
Verena Fernandes (56)	[ ]
Vedant Gowda (60)	[ ]
Ayush Gurav (61)	[ ]

9.2 Third-party components (not claimed)

VERIQO uses open-source software under permissive licences. These components remain the property of their authors and are excluded from any claim.

Component	Licence
FastAPI, SQLAlchemy, Pydantic, spaCy, pdfplumber, python-docx, tree-sitter, React, Vite, Tailwind CSS, Recharts	MIT
Sentence-Transformers, all-MiniLM-L6-v2 model, PyMongo	Apache 2.0
scikit-learn, PyTorch, Pygments	BSD-style
Python standard library (used only as training data for the clone classifier)	Python Software Foundation Licence

9.3 Disclosure of AI-assisted development

In line with the declaration in the project report, the team discloses that **significant portions of the source code and documentation were drafted with the assistance of an AI coding assistant (Anthropic's Claude), directed and reviewed by the team.** The design decisions, problem formulation and integration were carried out under the team's direction. As the extent of human authorship can affect copyright registration and the identification of inventors, the team requests the IPR Cell's guidance on how this should be reflected in any application.

9.4 Prior disclosures

The project has been described in the Major Project mid-term white book submitted to the Department. To the team's knowledge it has not been published, presented outside the Institute, or released in a public code repository. [Team to confirm before submission.]

10. Applications and Requested Action

10.1 Potential applications

- Hackathon and coding-competition organizers (colleges, companies, communities).
- Technical hiring assessments where take-home or timed coding tasks must be verified.
- University programming courses and lab examinations needing proof of authentic work.
- Developer portfolios with verified activity statistics ("Year in Code").

10.2 Request to the IPR Cell

1. Evaluate the work for copyright registration of the software.
2. Conduct or advise on a prior-art search for features N1–N4 and give an opinion on patentability, including Section 3(k).
3. Advise on a trademark clearance search for "VERIQO".
4. Advise on how AI-assisted development (Section 9.3) should be reflected in any application.
5. Consider the project for the programme mentioned in the Department's circular.

Annexure A — Clone Classifier: Data, Method and Results

Training corpus	450 Python files (CPython 3.13 standard library) and 103 JavaScript files (open-source npm packages). Files split 70/15/15 into training, validation and test sets before any pair was generated, so no file appears in two splits.
Pairs	4,379 labelled pairs; 622 fall below the comparison gate (fewer than 2 shared fingerprints or fewer than 50 tokens) and are never classified. Test set: 529 pairs, of which 361 are copies.
Copy types (positive)	Consistent identifier renaming with rewritten comments; reordered functions; heavy line edits; 15–25% and 25–50% partial copies inserted into unrelated files
Non-copies (negative)	Unrelated file pairs; different files from the same package (hard negatives)
Features	Containment in each direction, Jaccard similarity, syntax-tree n-gram Jaccard, size ratio, log of shared fingerprints
Model	Histogram gradient boosting with monotonic constraints (validation average precision 0.987 vs 0.983 for logistic regression)
Threshold	0.71 — the lowest value with validation false-positive rate ≤ 2%

Test-set result	Precision	Recall	False-positive rate
Trained classifier	0.982	0.906	0.036
Baseline (containment ≥ 0.5)	0.991	0.590	0.012
Python files only	0.993	0.903	0.013
JavaScript files only	0.934	0.919	0.308 (4 of 13)

Copy type (test set)	Detected
Reordered functions	98.0%
Heavy line edits	96.3%
Renamed identifiers and rewritten comments	91.7%
25–50% partial copy	87.5%
15–25% partial copy	80.8%

Limitations

- Copies are synthetic transformations of real code, so the figures are an upper bound until validated on labelled data from a real event.
- JavaScript data is small and its false-positive rate is not yet acceptable; more data is needed before JavaScript-heavy use.
- Some same-package negatives genuinely share code, so part of the false-positive rate is label noise.
- No AI-generated-code detector is claimed; telemetry is used instead, and its coverage is shown because the extension can be avoided.